

Mathematics for AI/ML

Revision Notes — Module 2

AI/ML Engineering Revision Journey

Topics Covered

1. Linear Algebra
2. Probability & Statistics
3. Calculus
4. Optimization

Focus: building intuition — understanding *why* these concepts power Machine Learning, Deep Learning, and modern AI systems, not just memorizing formulas.

1 Linear Algebra

1.1 Scalars, Vectors, Matrices & Tensors

These are the basic data containers used to represent everything a model works with — from a single number to an entire batch of images.

- **Scalar**: a single number, e.g. a learning rate 0.01 (0-dimensional).
- **Vector**: an ordered list of numbers, e.g. a word embedding [0.2, -1.3, 0.7] (1-dimensional).
- **Matrix**: a 2D grid of numbers, e.g. a dataset of rows (samples) × columns (features) (2-dimensional).
- **Tensor**: a generalization to n dimensions, e.g. a batch of colour images (batch × height × width × channels).

AI/ML Application: Neural network weights, image data, and transformer embeddings are all stored as tensors.

1.2 Vector & Matrix Operations

Vector operations

- Addition/subtraction: element-wise, vectors must be the same size.
- Scalar multiplication: scales every element, e.g. $3 \cdot [1, 2] = [3, 6]$.
- Vector norm (magnitude): $\|v\| = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$ — measures vector length.

Matrix operations

- Addition/subtraction: element-wise, matrices must match in shape.
- Matrix multiplication: rows of A are combined with columns of B; requires the number of columns of A to equal the number of rows of B.
- Transpose (A^T): flips rows and columns.
- Identity matrix (I): the matrix equivalent of the number 1; $A \cdot I = A$.
- Inverse (A^{-1}): the matrix that undoes A, so $A \cdot A^{-1} = I$ (only exists for square, non-singular matrices).

AI/ML Application: A dense/linear layer computes output = $W \cdot x + b$, which is just a matrix-vector multiplication followed by a vector addition.

1.3 Dot Product & Cosine Similarity

Dot product: $a \cdot b = a_1b_1 + a_2b_2 + \dots + a_nb_n$

Cosine similarity: $\cos(\theta) = (a \cdot b) / (\|a\| \cdot \|b\|)$

The dot product measures how much two vectors point in the same direction. Cosine similarity normalizes this by the vectors' lengths, giving a score between -1 (opposite) and 1 (identical direction), independent of magnitude.

AI/ML Application: Cosine similarity is the standard way to compare word/sentence embeddings and to power semantic search and recommendation systems (e.g. in RAG pipelines).

1.4 AI Applications in Neural Networks & Embeddings

- Forward pass of a layer: $z = W \cdot x + b$, followed by a non-linear activation.
- Embeddings represent words, images, or users as dense vectors so that similar items land close together in vector space.
- Attention in Transformers is built entirely from matrix multiplications (Query, Key, Value projections) and dot-product similarity between tokens.
- Eigenvectors/eigenvalues underpin techniques like PCA, used for dimensionality reduction.

2 Probability & Statistics

2.1 Probability Rules

- Probability of an event: $P(A)$ is between 0 and 1 (0 = impossible, 1 = certain).
- Complement rule: $P(\text{not } A) = 1 - P(A)$.
- Addition rule (mutually exclusive): $P(A \text{ or } B) = P(A) + P(B)$.
- Addition rule (general): $P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$.
- Multiplication rule (independent events): $P(A \text{ and } B) = P(A) \cdot P(B)$.

2.2 Conditional Probability

$$P(A | B) = P(A \text{ and } B) / P(B)$$

Conditional probability is the probability of A occurring given that B has already happened. It is the foundation of Bayes' Theorem:

$$P(A | B) = [P(B | A) \cdot P(A)] / P(B)$$

AI/ML Application: Naive Bayes classifiers, spam filters, and probabilistic language models rely directly on conditional probability and Bayes' theorem.

2.3 Mean, Median & Standard Deviation

- **Mean** (average): sum of all values divided by the count — sensitive to outliers.
- **Median**: the middle value when data is sorted — robust to outliers.
- **Standard deviation** (σ): measures how spread out values are around the mean; $\sigma = \sqrt{(1/n) \cdot \sum (x_i - \text{mean})^2}$.

AI/ML Application: Feature normalization/standardization (z-score scaling) uses the mean and standard deviation so that features contribute equally during training.

2.4 Variance & Outlier Detection

$$\text{Variance: } \sigma^2 = (1/n) \cdot \sum (x_i - \text{mean})^2$$

Variance is the average of squared deviations from the mean; standard deviation is its square root, expressed in the same units as the original data. Points that fall far outside the typical spread (commonly beyond 2-3 standard deviations, or outside the IQR-based whiskers) are flagged as outliers.

AI/ML Application: Understanding variance is essential for the bias-variance tradeoff: high-variance models overfit to noise in the training data.

3 Calculus

3.1 Functions & Derivatives

A function maps an input to an output, e.g. $y = f(x)$. The derivative $f'(x)$ measures how fast the output changes as the input changes — the slope of the function at a point.

$$f'(x) = \lim_{h \rightarrow 0} [f(x+h) - f(x)] / h$$

- Power rule: $d/dx (x^n) = n \cdot x^{n-1}$
- Chain rule: $d/dx f(g(x)) = f'(g(x)) \cdot g'(x)$ — the backbone of backpropagation.

3.2 Partial Derivatives

When a function depends on multiple variables (as loss functions in ML do, on every weight), a partial derivative measures the rate of change with respect to just one variable, holding all others constant. The collection of all partial derivatives is the **gradient**, written ∇f .

$$\nabla f = [\partial f / \partial w_1, \partial f / \partial w_2, \dots, \partial f / \partial w_n]$$

3.3 Gradient Descent

Gradient descent is an iterative algorithm that updates parameters in the direction opposite to the gradient, since the gradient points toward the steepest increase of the loss function.

$$w_{\text{new}} = w_{\text{old}} - \alpha \cdot \nabla f(w_{\text{old}})$$

Here α is the learning rate, controlling how big a step is taken at each iteration.

AI/ML Application: Every neural network is trained by repeatedly applying gradient descent to reduce the loss function.

3.4 Cost Functions

A cost (or loss) function quantifies how far the model's predictions are from the true values. The goal of training is to find parameters that minimize this function.

- Mean Squared Error (regression): $MSE = (1/n) \cdot \sum (y_{\text{pred}} - y_{\text{true}})^2$
- Binary Cross-Entropy (classification): measures the divergence between predicted and true class probabilities.
- Categorical Cross-Entropy: the multi-class extension, used with softmax outputs.

3.5 Backpropagation

Backpropagation efficiently computes the gradient of the loss with respect to every weight in a network by applying the chain rule layer-by-layer, moving backward from the output layer to the input layer. Each layer's gradient is reused to compute the gradients of the layers before it, avoiding redundant computation.

AI/ML Application: Backpropagation + gradient descent together form the training loop of virtually every deep learning model.

4 Optimization

4.1 Loss Functions (recap in optimization context)

Optimization is the process of searching for the parameter values that minimize the loss function. The choice of loss function shapes the entire optimization landscape and what the model ultimately learns.

4.2 Gradient Descent Variants

- **Batch Gradient Descent:** uses the entire dataset to compute each gradient step — stable but slow.
- **Stochastic Gradient Descent (SGD):** uses one random sample per step — fast but noisy.
- **Mini-batch Gradient Descent:** uses a small batch (e.g. 32-256 samples) per step — the standard in practice, balancing speed and stability.

4.3 Learning Rate

The learning rate (α) controls the step size in each parameter update.

- Too high: the optimizer can overshoot the minimum and diverge or oscillate.
- Too low: training becomes very slow and may get stuck in a shallow local minimum.
- Learning rate schedules/decay gradually reduce α over training for finer convergence.

4.4 Momentum, RMSProp & Adam

Momentum

Momentum accumulates a running average of past gradients, helping the optimizer keep moving in a consistent direction and pass through small local bumps or noisy gradients, similar to a ball rolling downhill.

$$v = \beta \cdot v_{\text{prev}} + (1 - \beta) \cdot \nabla f(w); w_{\text{new}} = w_{\text{old}} - \alpha \cdot v$$

RMSProp

RMSProp adapts the learning rate for each parameter individually, dividing by a running average of the magnitude of recent gradients. This helps parameters with large, noisy gradients take smaller steps, and parameters with small gradients take relatively larger steps.

Adam (Adaptive Moment Estimation)

Adam combines the ideas of Momentum and RMSProp: it keeps a running average of both the gradient (first moment) and the squared gradient (second moment), then uses both to adapt the step size per parameter. It is the most widely used optimizer in deep learning today due to its fast, stable convergence with minimal tuning.

AI/ML Application: Optimizer choice (SGD, RMSProp, Adam, AdamW, etc.) directly affects how quickly and reliably a deep learning model converges during training.
